

MULTI TP7

Siyuan CHEN

Xiru ZHANG

March 2024

Table des matières

B. Contrôleur DMA	2
Question B1	2
Question B2	2
Question B3	2
Question B4	2
Question B5	3
C. Architecture matérielle	3
Question C1	3
Question C2	3
D. Application logicielle	4
Question D1	4
Question D2	4
Question D3	4
Question D4	4
Question D5	4
Question D6	4
E. Pipeline logiciel	5
Question E1	5
Question E2	5
F. Traitement des erreurs	5
Question F1	5
Question F2	5
Question F3	5
G. Amélioration du parallélisme	6
Question G1	6

B. Contrôleur DMA

Question B1

Les registres adressables du contrôleur DMA sont les suivants :

- **SOURCE** : Adresse de base du tampon source, l'adresse de départ pour le transfert DMA source.
- **DEST** : Adresse de base du tampon de destination, l'adresse de départ pour le transfert DMA destination.
- **LENGTH/STATUS** : Longueur du transfert (en octets) / Statut. Une écriture spécifie la longueur du transfert DMA ; une lecture indique le statut du transfert en cours.
- **RESET** : Réinitialisation logicielle & acquittement IRQ.
- **NOIRQ** : Désactivation des IRQ quand non zéro.

L'adresse de base du segment associé au contrôleur DMA en tant que cible doit être alignée sur une frontière de bloc de 32 octets pour les raisons suivantes :

- **Efficacité des accès mémoire** : L'alignement sur une frontière de bloc de 32 octets assure que les accès mémoire sont réalisés de manière optimale, réduisant le temps nécessaire pour les transferts de données.
- **Optimisation de performance** : Cela permet d'éviter les accès mémoire supplémentaires qui seraient nécessaires pour traiter des données non alignées, améliorant la performance générale du système.

Question B2

burst : Désigne la longueur fixe pour une transaction en mode rafale. Cet argument définit le nombre maximal d'octets qui peuvent être transférés en une seule opération de rafale lors d'une transaction DMA.

Question B3

Le coprocesseur DMA se comporte donc comme une cible, puisqu'il doit être configuré par le système d'exploitation pour démarrer le transfert.

Il se conduit également comme un maître, puisqu'il est capable d'initier des transactions sur le bus pour lire ou écrire en mémoire.

Donc il est à la fois le maître et le cible. Pour ces deux scénarios, il a un comportement différent, donc a besoin de deux automates.

Question B4

La bascule `r_stop` dans le composant `PibusDma` est utilisée pour contrôler et synchroniser l'exécution des opérations entre deux machines : `TARGET_FSM` et `MASTER_FSM`. Voici ses principales fonctions et implications :

- **Initialisation** : `r_stop` est mis à `true`, ce qui indique que toutes les opérations de transfert DMA doivent être arrêtées ou ne pas démarrer.

- **Contrôle du Transfert** : Lorsque `r_stop` est mis à `false`, cela permet à la FSM maître (`MASTER_FSM`) de commencer le processus de transfert. Si `r_stop` est remis à `true` pendant un transfert, cela signale à la FSM de s'arrêter après l'achèvement du transfert en cours.
- **Synchronisation** : `r_stop` contrôle la synchronisation entre `TARGET_FSM`, qui gère les interactions avec le bus `PIBUS` et les registres internes, et `MASTER_FSM`.

Question B5

$A = \overline{END}$	$A' = END$
$B = GNT$	$B' = \overline{GNT}$
$C = \overline{LAST}$	$D = (ACK == \overline{WAIT}).LAST$
$D' = (ACK == \overline{WAIT}).\overline{LAST}$	$E = STOP$
$E = STOP$	$F = (ACK == ERROR)$
$G = (ACK == WAIT)$	$H = GNT$
$H = \overline{GNT}$	$I = \overline{LAST}$
$J = (ACK == \overline{WAIT}).LAST$	$J' = (ACK == \overline{WAIT}).\overline{LAST}$
$K = \overline{END}$	$L = (ACK == ERROR)$
$M = (ACK == READY)$	$N = (ACK == WAIT)$
$O = STOP$	$O' = \overline{STOP}$
$P = STOP$	$P' = \overline{STOP}$
$Q = STOP$	$Q' = \overline{STOP}$
$V = (ACK == READY)$	$W = (ACK == READY)$
$X = LAST$	$Y = LAST$

C. Architecture matérielle

Question C1

La longueur par défaut pour une rafale est 16 mots de 32 bits.

L'avantage en utilisant une grosse rafale : Réduire le cycle d'horloge en groupant les transferts des données, qui réduit la latence.

La conséquence sur le matériel est qu'il demande une plus grosse tampon mémoire pour stocker les données avant leur transfert.

Question C2

L'adresse de base du segment DMA est 0x93000000.

Son numéro de cible pour BCU est 6.

En tant que maître, son numéro pour BCU est 1. Il est connecté au porte 0 de ICU.

D. Application logicielle

Question D1

Cet appel-système est effectué par processeur.
Il est bloquant parce que la transaction n'est pas complet.

Question D2

La durée de construction d'une image est $1333385 - 718322 = 615063$ cycles.
La durée d'affichage est $1440012 - 1333385 = 106627$ cycles.

Question D3

La différence entre **fb_sync_write()** et **fb_write()** est que **fb_write()** a utilisé le matériel DMA pour transférer une gros de données, mais leur fonctionnements sont le même. **fb_completed()** est pour assurer que le transfert n'est pas interruptible jusqu'à prochaine interruption, cette comportement peut vérifier que le transfert est complet.

Question D4

La durée d'affichage avec DMA est 13217 cycles.

Question D5

Sur le bord gauche de l'image, on voit clairement qu'il existe une ligne diagonale apparaît en raison de l'incohérence des vitesses de traitement de l'image.
La cause de cette phénomène est que dans la fonction noyau **fb_write()**, si le périphérique DMA n'est pas disponible, le processeur **prod_id** s'attendre mais un autre processeur peut se disputer les ressources DMA. Chaque processeur attend plus ou moins longtemps les ressources DMA et le temps de traitement des transferts est différente également. Cela crée une incongruité visuelle.

Question D6

Pour ces deux appel-système, cette variable est utilisé dans leur fonctions noyau. Dans ces deux fonctions noyau, elle est utilisé pour attendre la terminaison d'un transfert, puis donne l'affichage d'écran par la fonction **_fb_write()**.
Cette variable est mise à 0 dans la fonction noyau **_isr_dma()**, qui signifie une interruption dma, et mise à 1 par la fonction noyau la fonction **_fb_write()**.
Elle est stocké dans le segment de noyau, avec l'adresse 0x81000008.

E. Pipeline logiciel

Question E1

Il faut assurer le dernier transfert est terminé par la fonction **fb_completed()**.

Question E2

Après modification du code, la durée d'affichage devient 3253 cycles, devient très bas par rapport 13217 cycles dans la question D4. Mais le nombre de cycles en total devient plus grand à cause de plus de itération dans la boucle (avant modification 5 itération, après modification 6 itération).

On peut dire bien qu'il a un gain d'affichage de $13217 - 3253 = 9964$ cycles par rapport à execution séquentielle, mais le gain en cycles total n'est pas efficace, voire inexistant.

F. Traitement des erreurs

Question F1

Le système d'exploitation interdit l'adresse source et destination appartient la zone de mémoire protégée pour la raison de sécurité. Cette zone de mémoire est utilisé pour empêche les programmes utilisateurs de corrompre les données dans les parties critiques du système en raison d'erreurs ou de comportements malveillants. La corruption de ces données peut entraîner un panique du système.

Question F2

Le mécanisme de signaler ce type erreur au programme utilisateur est suivant :

- Le DMA détecte l'erreur de bus, saute à l'état d'erreur
- Cet état provoque une interruption de DMA, cette interruption sauvegarde le status du processeur qui demande l'utilisation de DMA, puis signale ce transfert est terminé, puis reset le DMA.
- Pour donner le main au utilisateur, le noyau appel `_fb_completed()` pour attendre l'interruption DMA termine et affiche le message d'erreur au utilisateur pour signaler.

Question F3

L'adresse erronée est non définie : l'appel système était réussi avec retourne 0, mais l'image n'est pas généré.

L'adresse erronée appartient à la zone protégée : l'appel système `fb_write` signale l'erreur en utilisant la fonction noyau `_fb_write` qui retourne 1.

G. Amélioration du parallélisme

Question G1

Nous avons modifié le code de boot pour réaliser la parralélisme de processeurs. On

```
mfc0    $27,    $15,    1        # get the proc_id
addiu   $27,    $27,    1        # no more than 4 processors
li      $28,    0x40000         # stack size
mult    $27,    $28
mflo    $27

# initializes stack pointer
la      $27,    seg_stack_base
addu    $29,    $29,    $27
```

FIGURE 1 – code de boot

calcule l'espace nécessaire du processeurs correspondante par multiplication. Parce que tous les processeurs s'exécute la même programme pour traitement d'image, donc on ne modifie pas le reste.